

P1645R1

constexpr for <numeric> algorithms

Draft Proposal, 2019-05-14

This version:

<https://elbeno.github.io/isocpp/constexpr-numeric/constexpr-numeric.html>

Author:

Ben Deane (ben at elbeno dot com)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We have made many algorithms `constexpr` for C++20, but we seem to have overlooked the algorithms in `<numeric>`.

Table of Contents

- 1 Motivation
- 2 Assumptions
- 3 Algorithms in `<numeric>` that were apparently overlooked
- 4 Feature testing macro
- 5 Proposed wording relative to N4810
- 6 Thanks

References

Informative References

§ 1. Motivation

We have added `constexpr` to many parts of the standard library for C++20.

Notably, [P0879] *Constexpr for swap and swap-related functions* added `constexpr` to all functions in `<algorithm>` except `shuffle`, `sample`, `stable_sort`, `stable_partition`, `inplace_merge`, and functions accepting an `ExecutionPolicy`.

I believe LEWG's design intent is that the non-allocating, non-parallel algorithms be `constexpr`. However, there are algorithms in `<numeric>` that have been overlooked.

§ 2. Assumptions

[P0879] made the following assumptions:

- if an algorithm uses compiler intrinsics, then those intrinsics could be made `constexpr` by compiler vendors.
- if an algorithm uses assembly optimization, then that assembly could be turned into a `constexpr` compiler intrinsic.
- if an algorithm uses external functions, then those functions could be made `inline` and marked `constexpr` or could be replaced with intrinsics.

This proposal could make the same assumptions about implementation; however with the advent of `std::is_constant_evaluated` for distinguishing compile-time and runtime code, perhaps these assumptions are no longer necessary.

§ 3. Algorithms in `<numeric>` that were apparently overlooked

This proposal is to add `constexpr` to the following function templates in `<numeric>`, excepting the function templates that accept an `ExecutionPolicy`.

- `accumulate`
- `reduce`

- `inner_product`
- `transform_reduce`
- `partial_sum`
- `exclusive_scan`
- `inclusive_scan`
- `transform_exclusive_scan`
- `transform_inclusive_scan`
- `adjacent_difference`
- `iota`

§ 4. Feature testing macro

I propose the feature-testing macro `__cpp_lib_constexpr_numeric` to identify the presence of these `constexpr` forms.

§ 5. Proposed wording relative to N4810

Exactly as one would expect: add `constexpr` to the function templates listed above where they do not accept an `ExecutionPolicy`.

- Change **25.8 Header** `<numeric>` synopsis [**numeric.ops.overview**] as indicated:

```
template<class InputIterator, class T>
    constexpr T accumulate(InputIterator first, InputIterator last, T init);
template<class InputIterator, class T, class BinaryOperation>
    constexpr T accumulate(InputIterator first, InputIterator last, T init, B
```

```

template<class InputIterator>
constexpr typename iterator_traits::value_type
    reduce(InputIterator first, InputIterator last);
template<class InputIterator, class T>
constexpr T reduce(InputIterator first, InputIterator last, T init);
template<class InputIterator, class T, class BinaryOperation>
constexpr T reduce(InputIterator first, InputIterator last, T init, BinaryOperation op);

```

```

template<class InputIterator1, class InputIterator2, class T>
constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
                          InputIterator2 first2, T init);
template<class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
                          InputIterator2 first2, T init,
                          BinaryOperation1 binary_op1,
                          BinaryOperation2 binary_op2);

```

```

template<class InputIterator1, class InputIterator2, class T>
constexpr T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2,
                             T init);
template<class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
constexpr T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2,
                             T init,
                             BinaryOperation1 binary_op1,
                             BinaryOperation2 binary_op2);
template<class InputIterator, class T,
        class BinaryOperation, class UnaryOperation>
constexpr T transform_reduce(InputIterator first, InputIterator last,
                             T init,
                             BinaryOperation binary_op, UnaryOperation unary_op);

```



```
template<class InputIterator, class OutputIterator, class T,
        class BinaryOperation, class UnaryOperation>
constexpr OutputIterator transform_exclusive_scan(InputIterator first, In
                                                OutputIterator result,
                                                T init,
                                                BinaryOperation binary_
                                                UnaryOperation unary_op
```

```
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation>
constexpr OutputIterator transform_inclusive_scan(InputIterator first, In
                                                OutputIterator result,
                                                BinaryOperation binary_
                                                UnaryOperation unary_op

template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation, class T>
constexpr OutputIterator transform_inclusive_scan(InputIterator first, In
                                                OutputIterator result,
                                                BinaryOperation binary_
                                                UnaryOperation unary_op
                                                T init);
```

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator adjacent_difference(InputIterator first,
                                                InputIterator last,
                                                OutputIterator result);

template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator adjacent_difference(InputIterator first,
                                                InputIterator last,
                                                OutputIterator result,
                                                BinaryOperation binary_op);
```

```
template<class ForwardIterator, class T>
constexpr void iota(ForwardIterator first, ForwardIterator last, T value)
```

- Change **25.9.2 Accumulate** [**accumulate**] as the synopsis.

- Change **25.9.3 Reduce** [**reduce**] as the synopsis.
- Change **25.9.4 Inner product** [**inner.product**] as the synopsis.
- Change **25.9.5 Transform reduce** [**transform.reduce**] as the synopsis.
- Change **25.9.6 Partial sum** [**partial.sum**] as the synopsis.
- Change **25.9.7 Exclusive scan** [**exclusive.scan**] as the synopsis.
- Change **25.9.8 Inclusive scan** [**inclusive.scan**] as the synopsis.
- Change **25.9.9 Transform exclusive scan** [**transform.exclusive.scan**] as the synopsis.
- Change **25.9.10 Transform inclusive scan** [**transform.inclusive.scan**] as the synopsis.
- Change **25.9.11 Adjacent difference** [**adjacent.difference**] as the synopsis.
- Change **25.9.12 Iota** [**iota**] as the synopsis.

The feature-testing macro should also be updated.

- Add a line to Table 36, **17.3.1 General** [**support.limits.general**]/3 as indicated:

<code>__cpp_lib_constexpr_numeric</code>	[TBD]	<code><numeric></code>
--	-------	------------------------------

§ 6. Thanks

Thanks to Jan Wilmans, Joe Loser and Casey Carter.

§ References

§ Informative References

[P0879]

Antony Polukhin. [Constexpr for swap and swap related functions](https://wg21.link/P0879). 2017-12-29. URL: <https://wg21.link/P0879>